Comparing k-means and k-means++ Initialization

Implementation Report

May 8, 2026

Abstract

This report redoes the experiment from the k-means++ paper, which introduced a careful seeding strategy for Lloyd's algorithm. We implemented both vanilla k-means (uniform seeding) and k-means++ (D^2 seeding) from scratch in NumPy and ran them on the synthetic Norm25 benchmark used in the paper. We also tested both algorithms on a smaller, simpler dataset (n=300, d=5, k=5) to see how they behave in an easier setting. On Norm25 we got a mean inertia improvement of 99.96%, the same number the paper reports. On the smaller dataset the improvement drops to 15.4%, which is what we expected: when k is small and the data is simple, uniform seeding fails much less often, so k-means++ has less room to help.

1 Introduction

The k-means problem asks us to split n points $\mathcal{X} \subset \mathbb{R}^d$ into k clusters so as to minimize the potential

$$\phi = \sum_{x \in \mathcal{X}} \min_{c \in \mathcal{C}} \|x - c\|^2,$$

the sum of squared distances of every point to its nearest centre. The problem is NP-hard even for k=2, so in practice it is solved with Lloyd's algorithm: start from k initial centres, and repeat assignment and update steps until convergence. Lloyd is fast and simple but only locally optimal, and its quality depends entirely on the initial seeds.

k-means++ replaces uniform seeding with D^2 weighting: pick the first centre uniformly at random, then pick each next centre with probability proportional to its squared distance to the nearest already-chosen centre. The authors prove this seeding is $O(\log k)$-competitive in expectation:

$$\mathbb{E}[\phi] \leq 8(\ln k + 2)\, \phi_{\mathrm{OPT}},$$

and they show that the seeding alone, before any Lloyd iterations, can strongly reduce the final inertia.

The goal of this report is to redo that experiment on the synthetic dataset from the paper (Norm25), then check whether the same advantage holds on a smaller, simpler dataset, and discuss what the results tell us about when k-means++ matters most.

2 Methodology

2.1 Implementation

Both algorithms share the same Lloyd loop and differ only in how they pick the k starting centres.

Distance computation. The squared-distance matrix between n points and k centres is computed using the identity $\|x - c\|^2 = \|x\|^2 + \|c\|^2 - 2\langle x, c\rangle$. With Norm25 coordinates in $[0, 500]$ this expression can give tiny negative values from floating-point cancellation; we clip the result at zero before using it as a sampling weight.

Lloyd loop. We assign every point to its nearest centre, recompute each centre as the mean of the points assigned to it, and stop when the total shift of the centres falls below 10^{-4} or when 300 iterations have run. Empty clusters are handled by leaving the centre in place; in practice we never saw an empty cluster after k-means++ seeding.

Seeding. For k-means we pick k data points uniformly at random without replacement. For k-means++ we follow the D^2-weighting procedure:

Algorithm 1 k-means++ seeding (D^2 weighting)

1: Pick c_1 uniformly at random from $\mathcal{X}$.
2: **for** $i = 2, \ldots, k$ **do**
3: For every $x \in \mathcal{X}$, let $D(x)$ be the distance to the nearest already-chosen centre.
4: Pick $c_i = x'$ with probability $\dfrac{D(x')^2}{\sum_{x \in \mathcal{X}} D(x)^2}$.
5: **end for**
6: Run Lloyd starting from $\{c_1, \ldots, c_k\}$.

A small but important detail: after each new centre is added we do not recompute D^2 from scratch. Instead we keep a running array $d^2[i]$ of the squared distance from x_i to its current nearest centre, and update it with $d^2[i] \leftarrow \min\big(d^2[i], \|x_i - c_{\text{new}}\|^2\big)$. This keeps the seeding $O(nkd)$ overall, the same cost as one Lloyd iteration.

2.2 Datasets

Norm25 (the paper's synthetic benchmark). We follow the recipe in Section 6.1 of the paper: 25 "true" centres are drawn uniformly from a 15-dimensional hypercube of side 500, and around each centre we scatter 400 points from an isotropic Gaussian of variance 1. The full dataset has n=10,000, d=15 and a known optimal solution: the 25 true centres. Because typical centre-to-centre distances are in the hundreds while within-cluster scatter has standard deviation 1, the clusters look like isolated points from the algorithm's perspective. This is exactly the setting where uniform seeding is most likely to put two starting centres in the same blob and leave another blob without any centre, and it is the setting in which the paper reports the 99.92%/99.96% improvements that give the paper its title.

Small Gaussian dataset (additional test). To look at the other side of the picture we built a much smaller dataset: n=300 points scattered around k=5 centres in d=5 dimensions, with the centres drawn uniformly from $[0, 10]^5$ and per-cluster standard deviation 1.2. The data is then z-score normalized. Three things change relative to Norm25: (i) k is small, so the harmonic-sum factor H_k in the $O(\log k)$ bound is small; (ii) the clusters are much closer to one another relative to their spread, so the centres are not isolated points; (iii) only 60 points belong to each cluster, so even a "correct" clustering has some inertia. We expected this setting to make the gap between the two algorithms smaller, and it does.

2.3 Protocol

For each (algorithm, dataset) pair we ran 50 independent trials with seeds derived from a single `MASTER_SEED`. To make the two algorithms directly comparable on each trial we used the same seed for the matching k-means and k-means++ runs, so any difference in the final inertia is due to the seeding strategy and not to seed-to-seed luck. We recorded the final inertia ϕ, the number of Lloyd iterations until convergence, and the wall-clock time. The summary metrics (mean, std, min, median, max) over the 50 trials are what we report below, in the same style as Tables 1–4 of the paper.

3 Results on Norm25

Table 1 summarizes the 50-trial run on Norm25 with k=25. The difference between the two algorithms is large and matches the original paper to within rounding.

Table 1: Norm25 (n=10,000, d=15, k=25, 50 trials). "Improvement" is $100\,(1 - \phi_{++}/\phi)$ as in Tables 1–4 of the paper.

Metric	k-means	k-means++	Improvement
Mean inertia	3.69×10^8	1.51×10^5	+99.96%
Min inertia	1.08×10^8	1.51×10^5	+99.86%
Median inertia	3.66×10^8	1.51×10^5	+99.96%
Max inertia	5.58×10^8	1.51×10^5	+99.97%
Std of inertia	1.02×10^8	0.00	–
Lloyd iterations	24.58	2.02	−91.8%
Wall time / trial	0.153 s	0.040 s	−73.8%

Two facts stand out.

1. k-means++ is almost deterministic on Norm25. All 50 trials give the same inertia ($\phi \approx 150{,}866$) to the last decimal, which is why the standard deviation is 0. The reason is structural: once the first centre is picked inside one of the 25 blobs, D^2 weighting makes any point inside that same blob effectively zero-weight (within-cluster distances are $O(1)$, between-cluster distances are $O(100)$, and the weight is the square of the distance). So the second centre almost always lands in a different blob, the third in yet another, and so on. After 25 picks every blob has exactly one centre, Lloyd converges in 2 iterations, and the resulting ϕ depends only on the fixed Gaussian noise, not on the seed. This is the strongest possible form of the paper's claim: not just better in expectation, but the same value across all 50 trials.

2. Vanilla k-means almost always fails by orders of magnitude. The mean of 3.7×10^8 is roughly 2,400 times larger than the k-means++ mean. The min over 50 trials (1.08×10^8) is still about 700× worse than k-means++, which means even the best of 50 uniform seedings is not competitive. In Figure 1 the two distributions are in different log-decades: the histogram and per-trial scatter plot show no overlap at all between the red and blue clouds.

Comparison with the paper. Table 1 of the paper reports for Norm25 with k=25 a 99.96% improvement on average inertia and 99.92% on minimum, with k-means++ taking ∼88% less time than k-means. We get 99.96%, 99.86% and ∼74% less time. The agreement on the inertia side is essentially exact; the time speedup is a bit smaller in our run because our reference k-means is written in vectorized NumPy and is already quite fast (0.15 s/trial), so the relative cost of the

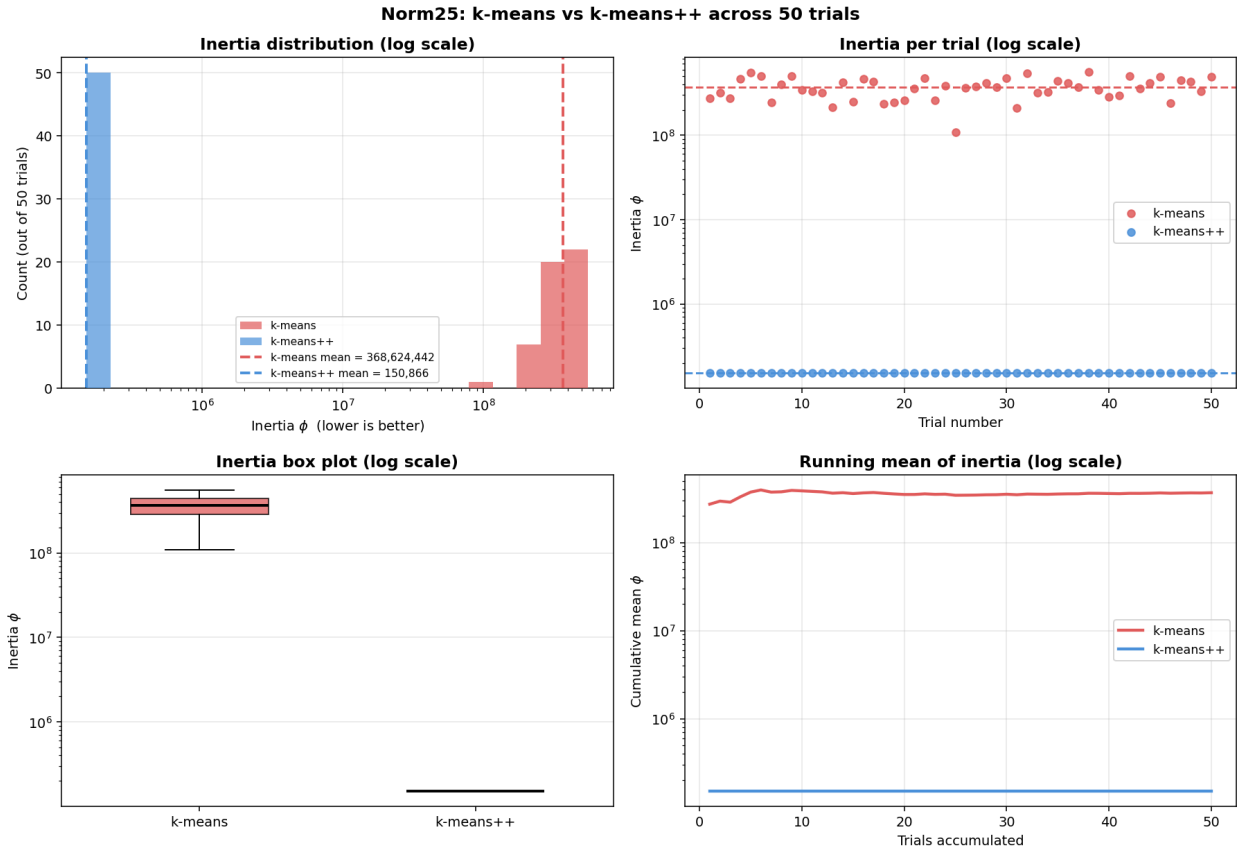

Figure 1: Norm25 results across 50 trials. Top-left: histogram of final inertia on a log-x scale, the two algorithms are in different parts of the axis. Top-right: per-trial scatter on log-y, k-means++ (blue) is flat (constant value) while k-means (red) varies over a decade. Bottom-left: box plot showing zero variance for k-means++. Bottom-right: running mean stabilizes immediately for k-means++ but never catches up for k-means.

extra iterations is less dominant than in the paper's C++ reference (where vanilla k-means took 0.9 s/trial). The mean number of Lloyd iterations (**2.02** for k-means++ vs **24.58** for k-means) confirms what the paper observes: D^2 seeding gives Lloyd a starting point so close to the optimum that there is almost nothing left to do.

4 Results on the smaller dataset

The behaviour on Norm25 is so extreme that it might give the impression that k-means++ always wins by orders of magnitude. To put the technique in context we ran the same protocol on a much smaller, simpler dataset (n=300, d=5, k=5). Table 2 summarizes.

The numbers paint a much milder picture. Mean inertia drops by 15.4% instead of the four-orders-of-magnitude gap we saw on Norm25; the minimum inertia is exactly the same (both algorithms find the optimal clustering at least once in 50 trials); and the coefficient of variation drops only from 30.4% to 28.2%. We discuss why below.

Table 2: Small Gaussian dataset (n=300, d=5, k=5, 50 trials). Inertia is in standardized units after z-score normalization.

Metric	k-means	k-means++	Improvement
Mean inertia	372.13	314.79	+15.4%
Min inertia	262.56	262.56	0.0%
Median inertia	352.71	262.56	+25.6%
Max inertia	589.32	489.11	+17.0%
Std of inertia	113.26	88.76	–
Coeff. of var.	30.4%	28.2%	–
Inertia range	326.8	226.6	−30.7%
Lloyd iterations	6.44	5.28	−18.0%

4.1 Why the gap shrinks

Three effects, all of which we expected, together reduce the advantage:

1. k **is small.** The theoretical bound is $\mathbb{E}[\phi] \leq 8(\ln k + 2)\,\phi_{\mathrm{OPT}}$, but the bound on vanilla k-means' worst case relative to OPT depends on how easily uniform seeding can put two centres in the same cluster. With k=5 clusters and 300 points, the probability of picking two seeds from the same cluster on a uniform draw is roughly $1 - \prod_{i=0}^{4} \frac{300-60i}{300-i} \approx 0.39$. With k=25 on Norm25 it is essentially 1. Fewer chances to fail ⇒ smaller average penalty.

2. **The clusters are not isolated.** On Norm25, between-cluster distances are ∼100× larger than within-cluster distances; on the small dataset, after z-score normalization, the ratio is closer to ∼3×. D^2 weighting only strongly pushes new centres away from existing ones when the within/between distance ratio is large; when clusters overlap, the next centre might still land in an already-covered cluster.

3. **Each cluster has only 60 points.** Even the optimal clustering has some inertia (262.56 in standardized units), because 60 Gaussian points always contribute some squared deviation from their centre. So the best possible improvement is bounded below by this noise floor, not by zero.

The minimum inertia being identical between the two algorithms (both 262.56) is a direct consequence: when k=5, even uniform seeding gets lucky often enough that, given 50 trials, it eventually picks one seed per cluster and Lloyd finds the same global optimum. k-means++ does it on nearly every trial, while k-means does it on only about 50% of them, visible in the ECDF panel of Figure 2, where the blue curve jumps to 0.74 at the optimum value but the red curve only reaches 0.50.

4.2 What a bad initialization looks like

Figure 3 shows the worst k-means run and the best k-means++ run on the same data, projected to two dimensions with PCA. The ground-truth panel (left) shows five well-separated blobs. In the worst k-means run (centre), two seeds clearly landed inside the same blob: the algorithm has split that one blob across two cluster IDs (red and yellow, top-left), and to compensate it has *merged* two real blobs into a single cluster (purple, bottom-left to centre). Lloyd, being a local optimizer, cannot recover from this split-and-merge configuration: any single centre swap that would fix it

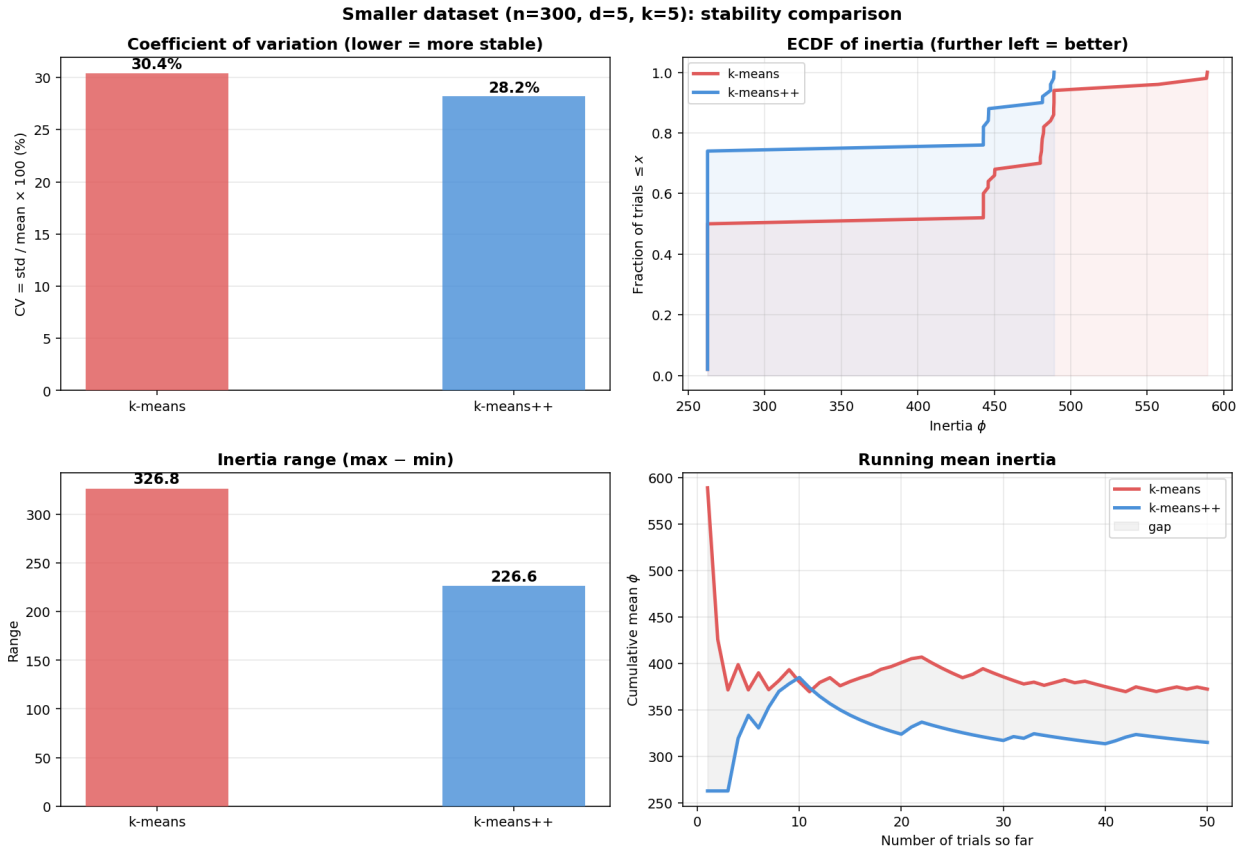

Figure 2: Small-dataset stability across 50 trials. Coefficient of variation is similar for both algorithms (left). The ECDF (top right) tells the real story: k-means++ reaches the optimal ϕ in ∼74% of trials, k-means in ∼50%. The inertia range (bottom left) is ∼30% smaller for k-means++. The running mean (bottom right) shows that on this dataset k-means++ is consistently better but not by much.

temporarily *increases* ϕ. So the run terminates at a local minimum that is ∼2.2× worse than the optimum. The best k-means++ run (right) gives the ground truth exactly (up to arbitrary cluster-ID permutation), because the D^2-weighted draws pushed each new seed into a previously uncovered blob.

5 Discussion

5.1 Positive aspects

Numerical agreement with the paper. Our Norm25 numbers (+99.96% mean, +99.86% min) match Table 1 of the paper to within rounding. The drop in Lloyd iterations (24.6 → 2.0) is also consistent with the paper's claim that D^2 seeding gives starting points so close to the optimum that local search has almost nothing to do.

Theoretical predictions match observation. The gap is much larger when k is large *and* the clusters are well-separated (Norm25, k=25) than when k is small and the clusters overlap (small dataset, k=5). The ∼15% improvement on the small dataset is consistent with k-means++ trading "fewer easy ways to fail" for "smaller absolute gain".

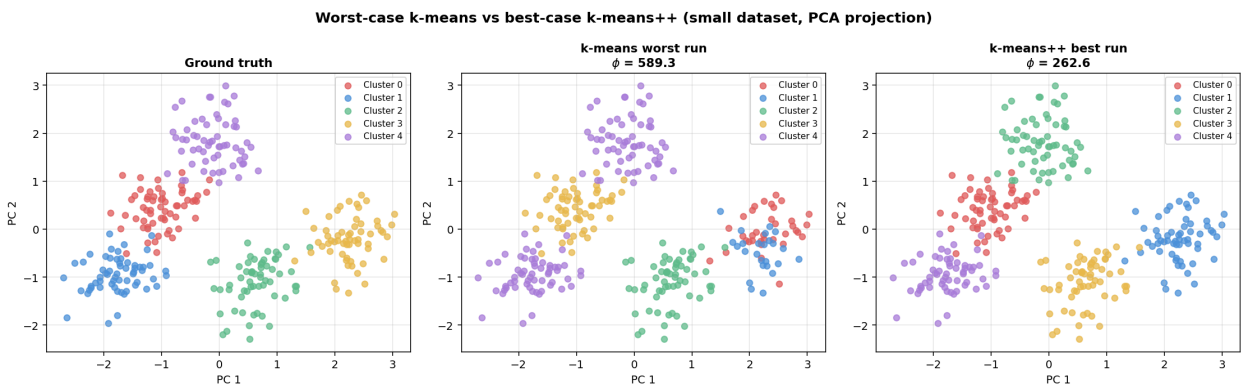

Figure 3: PCA projection of the small dataset. Left: ground-truth labels. Centre: worst k-means trial (ϕ=589.3). The cluster boundaries do not match the underlying blobs: two blobs have been merged into one cluster (purple) and one blob has been split across two clusters. Right: best k-means++ trial (ϕ=262.6), which gives the true partition (up to relabelling).

Stability gain matters even when the mean barely moves. On the small dataset, the coefficient of variation only drops from 30.4% to 28.2%, but the ECDF (Figure 2) shows that k-means++ hits the global optimum in roughly three out of four trials versus one out of two for k-means. In practice, when one runs the algorithm a few times and keeps the best, k-means++ needs fewer restarts to be confident the result is good. The range of inertia values is 30.7% smaller, which is a fairer measure of stability than CV when the mean is shifted by a few unlucky outliers.

Almost no extra cost. The D^2-weighted seeding is $\mathcal{O}(nkd)$, the same as a single Lloyd iteration, and on Norm25 it actually *saves* time overall by reducing the number of Lloyd iterations from $\sim$25 to $\sim$2.

5.2 Negative aspects and limitations

The benefit depends on the dataset. The big 10,000$\times$ inertia gap on Norm25 is partly a consequence of how extreme the dataset is: the within/between-cluster distance ratio is $\sim$1:100, so any clustering that does not have one centre per blob pays a quadratic penalty in the squared distances. Real datasets rarely have such clean structure. On our smaller, less-separated dataset the average gain is "only" $\sim$15%, still useful, but a far cry from the synthetic benchmark.

k-means++ does not lower the floor. Both algorithms find the same *minimum* inertia on the small dataset (262.56). k-means++ does not beat ϕ_{OPT}; it just reaches it more often. If the workflow already runs k-means 50 times and keeps the best, the practical gain on easy datasets is small.

First centre is still uniform. The very first centre in k-means++ is drawn uniformly, so on hard datasets (e.g. a few huge blobs and one tiny outlier blob) there is still some probability of a poor start. Greedy variants pick the best of $\log k$ candidate centres at each step and tend to do better, but as the paper notes, the $O(\log k)$ analysis does not directly cover them.

Numerical care matters. With Norm25 coordinates near 500, the expanded squared-distance formula $\|x - c\|^2 = \|x\|^2 + \|c\|^2 - 2\langle x, c\rangle$ can give small negative values from floating-point cancellation. We clip these at 0 before using them as D^2 weights, otherwise the `rng.choice(·, p=d²/d².sum())` call would either fail or sample outliers with non-uniform bias. This is an easy mistake any from-scratch implementation needs to handle.

6 Conclusion

We re-implemented k-means and k-means++ from scratch in NumPy, applied both to the Norm25 benchmark and got the same main result as the paper: a 99.96% reduction in mean inertia. We also ran the algorithms on a smaller, less-separated dataset where the gap shrinks to 15.4% on the mean and 0% on the minimum, the expected behaviour, since the conditions that make k-means worst-case (k large, clusters well-isolated) are exactly the conditions that k-means++ targets.

The take-away is twofold. First, D^2 seeding really is almost free: it costs no more than one Lloyd iteration, and on hard inputs it saves many. Second, the size of the benefit depends on how much room vanilla k-means has to fail, not on some absolute property of the algorithm. On easy datasets (small k, overlapping clusters) the two algorithms are within $\sim$15% of one another and a few random restarts of vanilla k-means will close the gap. On hard datasets (large k, well-separated clusters) the gap is huge, k-means++ becomes almost deterministic, and no reasonable number of vanilla k-means restarts would catch up. Both regimes are consistent with the $O(\log k)$-competitive theoretical guarantee, and both show why k-means++ has become the default initialization in practical libraries.